COMP1002/8802 – Lab 1 Worksheet

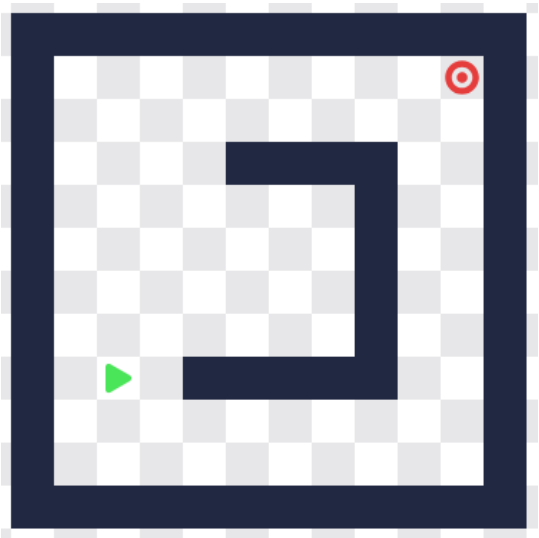
You should complete this worksheet during your lab/workshop session. Your tutor will be able to help with any problems or questions you have. Answers for the (marked) parts of this worksheet must be included in your portfolio assessment submission.

Part 1 – Maze Solving (marked)

The following website allows you to test out some different search algorithms in a simple grid-world environment: https://angeluriot.com/maze_solver/

In this environment, our agent (green triangle) can move up, down, left or right each step. Our goal is to reach the target square (red bullseye), but we cannot pass through solid wall tiles.

Tasks:
Replicate the maze layout below and run the **Breadth-First**, **Best-First (Greedy)** and **A*** search algorithms:



Note. Please make sure to revise the lecture slides if you are not sure how each of these search algorithms work.

For Best-First and A* search algorithms, the Manhattan distance to the target (i.e., the shortest number of steps needed to reach the target if all walls were ignored) is used as a heuristic evaluation function ***h(s)*** to help guide our search.

Questions:
How do these three search algorithms compare on the following aspects:

- **Path Length**
- **Algorithm runtime**
    - Note., the number of states checked by the algorithm (i.e., squares with blue circles) gives a general indication of the algorithms speed (less states checked = faster)

Think carefully about whether this result seems consistent with your understanding of how these search algorithms work. Provide a brief explanation for your observations.

Part 2 – Depth-First Search (marked)

While this website does not provide the option for a **depth-first** search algorithm, think about how this algorithm would perform for the same maze layout given in Part 1

Hint, assume states are ordered: North, East, South, West, in terms of selection priority.

Questions:

- What is the path that depth-first search would take to the target?
- Does depth-first search find an optimal path to the target?
- Does depth-first search take more or less time (i.e., checks more or less states) than each of the other search algorithms (Breadth-First, Best-First, and A*)?

Tasks:

Modify the maze layout given in Part 1, so that depth-first search returns a path that is longer than any of the other search algorithms.

Part 3 – Slow and Long (marked)

A general rule of thumb is that best-first search is often faster than other search algorithms, with the downside that it may give a non-optimal path. However, this is not always the case, as there are certain situations where best-first search finds a non-optimal path AND takes longer to run than A* search.

Tasks:

Design a new maze layout where best-first search is both slower than A* (i.e., checks more states) and returns a non-optimal path. You will need to think carefully about how these algorithms work and experiment with several different maze designs using the provided software.

Note. the bonus tasks for this worksheet focus on using Markov Decision Processes for planning problems. They should only be attempted after this technique has been covered in the lecture slides.

Bonus 1 – Game Show

You have been chosen to participate in your favourite TV game show “Markov Decision Prizes!”. The rules for the game are as follows:

- You are asked a series of questions across ten rounds
- If you answer the question correctly, your prize amount increases (given in the table below) but if you get it wrong you go home with nothing.
- At the start of each round, you can either stop and keep your current prize amount, or you can risk answering the question.

Based on the results of past contestants, we can also estimate the probability of you correctly answering the question in each round (given in the table below):

Round	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Prize Amount	$1k	$3k	$8k	$16k	$32k	$64k	$128k	$250k	$500k	$1m
Probability of Correct Answer	99%	90%	80%	70%	60%	50%	40%	30%	20%	10%

Tasks:
Design a Markov Decision Process diagram that represents this game show.

Hint, in this case we want as much money as possible, so winning money should have a negative cost value

Questions:
- Can you give an estimate for when it would be most sensible to stop answering questions?
- What strategies did you use to make your decision?
- How would changing the prize amounts or probability of a correct answer affect your decision?

Bonus 2 – Solar Robot

Consider the following scenario:
- There is a robot that operates on a steep slope
- The slope has three levels (low, medium and high)
- Each minute, the robot has the option to spin its wheels (or not spin them)
    - If the robot spins its wheels, it will move up the slope one level (or stays at high level if already there)
    - If the robot doesn't spin its wheels, then it will slide down the slope one level (or stays at low level if already there)
- Spinning its wheels uses up 1kw of the robot's energy
- The robot is equipped with solar panels that provide it will 3kw of energy every minute, but only if the robot is on the high or medium levels of the slope

Tasks:
Design a Markov Decision Process that represents this situation, and use the value iteration algorithm to determine the policy that provides the robot with the maximum amount of energy
Note. You can stop after three or four value iterations, as the policy will not change after this.

Hint, in this case we want as much energy as possible, so when a robot gains energy this should be a negative cost (i.e., -3) and when it spends energy this should be a positive cost (i.e., +1)

Bonus 3 – Solar Robot v2

Consider the following variations to how the solar robot moves in the above:
- If the robot spins its wheels, there is a 30% chance that it will move up the slope one level (or stays at high level if already there) and a 70% chance that it stays at its current level
- If the robot doesn't spin its wheels, there is a 40% chance that it will slide down the slope one level (or stays at low level if already there) and a 60% chance that it stays at its current level

Tasks:
Design a Markov Decision Process that represents this new probabilistic situation, and use the value iteration algorithm to determine the policy that provides the robot with the maximum amount of energy
Note. You can stop after three or four value iterations, as the policy will not change after this.

Bonus 4 – Slope Climbing Robot

Consider the following scenario:

- There is a robot that operates on a steep slope
- The slope has four levels (low, medium, high and top)
- Each minute, the robot has the option to spin its wheels either slowly or rapidly
    - If the robot spins its wheels slowly, there is a 30% chance that it will move up the slope one level and a 70% chance that it will fall to the low level
    - If the robot spins its wheels rapidly, there is a 50% chance that it will move up the slope one level and a 50% chance that it will fall to the low level
- Spinning its wheels slowly uses up 1kw of the robot's energy, and spinning its wheels rapidly uses 2kw of the robot's energy

Tasks:
Design a Markov Decision Process that represents this situation, and use the value iteration algorithm to determine the policy that moves the robot to the top of the slope with the minimum expected energy usage.
Note. Unlike the previous questions, you will need to run the value iteration algorithm for many iterations before you arrive at the correct policy…